

Prototype Design Pattern — Complete Guide (Beginner → Advanced, Python 3)

Every code example in this guide was actually run to confirm it behaves exactly as described — including a genuinely important, verified detail: whether `__init__` runs again when you clone an object (Section 6).

1. The Big Idea in One Line

Prototype means: **instead of building a new object from scratch, copy an existing, already-configured one, and only change what's different.**

Think of a photocopier. If you need ten copies of a filled-out form, you don't retype the whole form ten times from a blank template — you fill it out once, then photocopy it. Each copy starts out identical to the original, and you only need to write in whatever's different (like a name field) on each one.

2. Formal Definition (the one interviewers expect)

Specify the kinds of objects to create using a prototypical instance, and create new objects by copying this prototype.

The key shift: creation stops being "call a constructor and pass in all the arguments" and becomes "ask an existing object to duplicate itself." The new object's starting state comes from copying, not from re-running construction logic.

3. Why Do We Even Need This?

Three real situations where this matters:

1. **Construction is expensive.** If building an object involves a database query, a network call, loading a large file, or heavy computation, re-running that work for every new object is wasteful. Cloning an already-built object skips all of that (verified in Section 6 — cloning genuinely does not re-run `__init__`).
2. **You want "mostly the same, with small tweaks."** Spawning 50 goblins in a game that are identical except for position, or generating variant configurations from one well-tested baseline, is naturally a "copy and tweak" operation, not a "build every field again" operation.
3. **You want to create objects without depending on their concrete class.** A function that just calls `some_object.clone()` doesn't need to know or import the object's actual class, its constructor signature, or how many arguments it takes — it works polymorphically on *anything* that knows how to copy itself. This is the same "decouple from concrete classes" spirit as the Factory patterns, but solved by copying instead of by construction.

4. The Structure — Three Participants

1. **Prototype** (interface/abstract class) — declares a `clone()` method (or, in idiomatic Python, relies on the built-in copy protocol described in Section 5).

2. **ConcretePrototype** — implements cloning, typically by copying itself.
3. **Client** — asks a prototype object to clone itself, without ever needing to know its concrete class or how to construct one from scratch.

5. The Basic Implementation (Classic GoF style)

```
python

from abc import ABC, abstractmethod
import copy

class Prototype(ABC):
    @abstractmethod
    def clone(self):
        pass

class ConcreteShape(Prototype):
    def __init__(self, shape_type, x, y, color):
        self.shape_type = shape_type
        self.x = x
        self.y = y
        self.color = color

    def clone(self):
        return copy.deepcopy(self)

    def __repr__(self):
        return f"ConcreteShape({self.shape_type}, x={self.x}, y={self.y}, color={self.color})"

circle = ConcreteShape("circle", 10, 20, "red")
circle_clone = circle.clone()
circle_clone.x = 999

print(circle)          # ConcreteShape(circle, x=10, y=20, color=red)
print(circle_clone)    # ConcreteShape(circle, x=999, y=20, color=red)
```

(Verified — output matches exactly.) Whatever code calls `circle.clone()` never had to know it was working with a `ConcreteShape`, or pass in `shape_type`, `x`, `y`, and `color` again — it just asked the object to duplicate itself.

6. The Most Important Python-Specific Fact: `copy.deepcopy()` Does NOT Re-run `__init__`

This is genuinely worth knowing cold, because it's *the* reason Prototype is valuable for expensive objects, and it's rarely stated explicitly.

python

```
class ExpensiveObject:
    init_calls = 0
    def __init__(self, value):
        ExpensiveObject.init_calls += 1
        print(f"Running EXPENSIVE __init__ (call #{ExpensiveObject.init_calls})")
        self.value = value

obj = ExpensiveObject(42)
# Running EXPENSIVE __init__ (call #1)...

cloned = copy.deepcopy(obj)
# (nothing printed - __init__ did NOT run again)

print(ExpensiveObject.init_calls) # still 1
print(cloned.value)                # 42 - the state still came through correctly
print(cloned is obj)               # False - it's a genuinely separate object
```

(Verified.) Python's default copying mechanism reconstructs the object by allocating a new instance directly and copying its `__dict__` across — it never calls `__init__` a second time. This is the same underlying mechanism you may have seen if you covered Singleton's pickling behavior — `copy` and `pickle` both build objects through a low-level reconstruction path, not by calling the constructor. Practically, this means: if your `__init__` does something slow (loading a model, querying a database, opening a connection), cloning a fully-built object is genuinely faster than constructing a new one — not just theoretically, but literally, measurably fewer function calls.

7. The Gotcha Every Interview Will Probe: Shallow Copy vs. Deep Copy

(This is the diagram shown above in chat.)

Python's `copy` module gives you two very different levels of cloning, and mixing them up is the single most common Prototype-related bug.

python

```
class Monster:
    def __init__(self, name, health, abilities, inventory):
        self.name = name
        self.health = health
        self.abilities = abilities # a list - mutable!
        self.inventory = inventory # a list - mutable!

original = Monster("Goblin", 30, ["bite", "scratch"], ["dagger"])
```

`copy.copy()` — shallow copy

```
python

shallow_clone = copy.copy(original)
shallow_clone.name = "Goblin Clone"
shallow_clone.abilities.append("poison bite")

print(original.abilities)
# ['bite', 'scratch', 'poison bite'] <- the ORIGINAL changed too!

print(shallow_clone.abilities is original.abilities) # True — same list object
```

(Verified — the original's list really does change.) A shallow copy duplicates the *outer* object, but any mutable attributes inside it (lists, dicts, other objects) are still the exact same shared objects as the original — only the reference got copied, not the thing it points to. Change the clone's list, and you're changing the original's list too, because there was only ever one list.

`copy.deepcopy()` — deep copy

```
python

original2 = Monster("Goblin", 30, ["bite", "scratch"], ["dagger"])
deep_clone = copy.deepcopy(original2)
deep_clone.abilities.append("fireball")

print(original2.abilities)
# ['bite', 'scratch'] <- unaffected!

print(deep_clone.abilities is original2.abilities) # False — genuinely separat
```

(Verified.) `deepcopy` recursively copies everything — the outer object *and* every mutable object nested inside it — so the clone is completely independent. This is almost always what you actually want when implementing Prototype, which is why `clone()` implementations typically call `copy.deepcopy(self)` rather than `copy.copy(self)`.

Interview-ready one-liner: "Shallow copy duplicates the container but shares the contents; deep copy duplicates the container and recursively duplicates the contents too."

8. Advanced: Overriding `__deepcopy__` to Choose What Gets Shared

Sometimes "deep-copy absolutely everything" is wrong too. Imagine a `Monster` that holds a reference to a `GameAssetLoader` — an expensive, shared resource (textures, sounds) that every monster of the same type should *reuse*, not duplicate. You want to deep-copy the monster's own state (abilities, inventory) while deliberately *sharing* the asset loader:

python

```
class GameAssetLoader:
    load_calls = 0
    def __init__(self):
        GameAssetLoader.load_calls += 1
        print(f>Loading game assets (expensive, call #{GameAssetLoader.load_calls})
        self.textures = {"goblin": "goblin.png"}

class MonsterWithAssets:
    def __init__(self, name, abilities, asset_loader):
        self.name = name
        self.abilities = abilities
        self.asset_loader = asset_loader # expensive, meant to be shared

    def __deepcopy__(self, memo):
        new_monster = MonsterWithAssets.__new__(MonsterWithAssets)
        memo[id(self)] = new_monster
        new_monster.name = self.name
        new_monster.abilities = copy.deepcopy(self.abilities, memo) # duplicate
        new_monster.asset_loader = self.asset_loader # deliberate sharing
        return new_monster

loader = GameAssetLoader() # "Loading game assets..." prints once
goblin = MonsterWithAssets("Goblin", ["bite"], loader)
goblin_clone = copy.deepcopy(goblin)
goblin_clone.abilities.append("poison")

print(goblin.abilities) # ['bite'] - unaffected
print(goblin_clone.abilities) # ['bite', 'poison']
print(goblin_clone.asset_loader is goblin.asset_loader) # True - shared, not copied
print(GameAssetLoader.load_calls) # still 1 - never loaded again
```

(Verified — the asset loader is genuinely shared, and it never gets loaded a second time.)

This is Prototype at its most powerful: you get complete control over *exactly* what "cloning" means for your object — duplicate what needs to be independent, share what's expensive and safe to share. `memo` is a dictionary `deepcopy` passes around to avoid infinite loops on circular references and to avoid copying the same nested object twice — always thread it through when you call `copy.deepcopy(x, memo)` inside your own `__deepcopy__`.

9. Prototype Registry (Managing a Set of Named Prototypes)

A very common real-world extension: keep a small library of pre-configured "template" objects, and let callers request a clone by name instead of constructing one from scratch every time.

python

```
class MonsterRegistry:
    _prototypes = {}

    @classmethod
    def register(cls, name, prototype):
        cls._prototypes[name] = prototype

    @classmethod
    def create(cls, name):
        prototype = cls._prototypes.get(name)
        if prototype is None:
            raise ValueError(f"No prototype registered for: {name}")
        return copy.deepcopy(prototype)

goblin_prototype = Monster("Goblin", 30, ["bite"], ["dagger"])
MonsterRegistry.register("goblin", goblin_prototype)

g1 = MonsterRegistry.create("goblin")
g2 = MonsterRegistry.create("goblin")
g1.name = "Goblin #1"
g1.abilities.append("rage")

print(g1)    # Monster(name='Goblin #1', health=30, abilities=['bite', 'rage'],
print(g2)    # Monster(name='Goblin', health=30, abilities=['bite'], inventory=
print(g1 is g2) # False

MonsterRegistry.create("dragon")
# ValueError: No prototype registered for: dragon
```

(Verified — including the error for an unregistered name.) This is structurally the same idea as the registry-based Factory Method and Abstract Factory variants you've already seen — a dictionary of "recipes" keyed by name — except here the "recipe" is a fully-built object to clone, rather than a class to instantiate.

10. Real-World Examples You Can Quote

- `copy.copy()` / `copy.deepcopy()` themselves are literally Python's built-in Prototype support — there's no need to reinvent a `clone()` protocol from scratch in most Python code.
- `list.copy()` and `dict.copy()` — the standard library's own built-in collections ship shallow-copy methods (verified: `list.copy()` duplicates the top-level list but still shares nested mutable objects, exactly like `copy.copy()`).

- `sklearn.base.clone()` in scikit-learn — used constantly in cross-validation and hyperparameter search. It builds a fresh, unfitted copy of an estimator with the same hyperparameters (verified: cloning a `LogisticRegression(C=0.5)` preserves `C=0.5` but drops any "fitted" attributes). Worth noting precisely: sklearn's `clone()` isn't a blind `deepcopy` of all instance state — it reconstructs a new instance from just the estimator's *configured parameters*, which is exactly why fitted attributes don't carry over. It's a great example of a *deliberately selective* clone, in the same spirit as Section 8.
- Java's `Cloneable` interface / `Object.clone()` — the classic textbook reference point for this pattern in other languages, if it comes up by name in an interview.

11. Prototype vs. Factory Method vs. Builder

You now have the full creational-pattern toolkit — here's Prototype's place in it:

- **Factory Method** — decide *which class* to instantiate, then build it from scratch via its constructor.
- **Builder** — construct *one* complex object step by step, through a multi-stage assembly process.
- **Prototype** — skip construction entirely for the parts that are already correct; copy an existing object and only change what's different.

A good mental test: if creating an equivalent object from scratch would be *expensive* or would require *re-deriving state you already have on hand*, reach for Prototype instead of any of the other three.

12. Prototype and "Class Explosion"

Factory Method's Section on subclassing showed how you can end up with one subclass per configuration variant. Prototype offers a different way out of that problem: instead of writing a new subclass for every variant (`EliteGoblin`, `WeakGoblin`, `FastGoblin`...), you keep one well-configured prototype instance around and clone-and-tweak it for each variant:

```
python

elite_goblin = MonsterRegistry.create("goblin")
elite_goblin.name = "Elite Goblin"
elite_goblin.health = 60
```

No new class was written. This trades "a class per variant" for "a runtime-configured object per variant" — worth explicitly contrasting with Factory Method's subclassing approach in an interview if asked how to avoid class explosion.

13. Pros and Cons

Pros

- Avoids repeating expensive construction work — cloning skips `__init__` entirely (Section 6).
- Decouples client code from concrete classes — cloning works polymorphically on anything with a `clone()` method, without needing to know constructor signatures.
- Naturally supports "mostly the same, slightly different" object creation without a subclass per variant.
- Gives fine-grained control over what gets duplicated vs. shared, via custom `__deepcopy__` (Section 8).

Cons

- Shallow vs. deep copy is a real, easy-to-hit source of bugs if you're not deliberate about it (Section 7) — silently sharing mutable state between "independent" objects is a subtle, hard-to-debug class of error.
- Objects with complex internal references (circular references, external resources like open file handles or network sockets) need careful custom `__deepcopy__` logic — a naive deep copy can break or duplicate things that should never be duplicated.
- If most of your objects are cheap to construct and don't share meaningful state, Prototype adds a layer of indirection for no real benefit — plain construction is simpler.

14. When to Use It (and When Not To)

Use it when:

- Construction is genuinely expensive (network calls, file I/O, heavy computation, large object graphs).
- You need many objects that are mostly identical, differing only in a few fields.
- You want object creation code that works polymorphically without knowing concrete classes or constructor signatures.

Be cautious when:

- Your objects are cheap and simple to construct — a plain constructor call is clearer and needs no thought about shallow vs. deep copying.
- Your object holds external resources (file handles, network connections, locks) — these usually should *not* be blindly duplicated, and need explicit, careful handling in a custom `__deepcopy__` (or should simply be excluded from cloning entirely, the way `sklearn.clone()` excludes fitted state).

15. Interview Question Bank

Q1: What problem does the Prototype pattern solve? It avoids repeating expensive or repetitive object construction by copying an existing, already-configured object instead of building a new one from scratch, and it lets client code create new objects without knowing their concrete class.

Q2: What's the difference between shallow copy and deep copy? Shallow copy duplicates the outer object but keeps any nested mutable objects (lists, dicts, other objects) as shared references with the original — mutating one affects the other. Deep copy recursively duplicates everything, so the result is fully independent (verified in Section 7).

Q3: Does cloning an object re-run its `__init__` method? No — Python's default `copy`/`deepcopy` mechanism reconstructs the object directly and copies its state across without calling `__init__` again. This is precisely why Prototype is valuable for objects with expensive construction logic (verified in Section 6).

Q4: How would you implement Prototype in Python? Lead with `copy.deepcopy()` as the default, idiomatic choice — it already implements the core of this pattern. Mention overriding `__deepcopy__` when you need fine-grained control over what gets duplicated vs. shared (Section 8), and the classic abstract `Prototype` + `clone()` interface if the interviewer wants the textbook GoF shape.

Q5: When would you override `__deepcopy__` instead of just calling `copy.deepcopy(self)`? When some part of the object's state is expensive and safe to share across clones (like a shared asset loader or a read-only config object) — you want to deep-copy the parts that must be independent while deliberately keeping a shared reference to the parts that shouldn't be duplicated (Section 8).

Q6: What's a real-world Python example of this pattern outside your own code? `list.copy()` / `dict.copy()` in the standard library, or `sklearn.base.clone()`, which builds a fresh, unfitted estimator from just an existing estimator's hyperparameters rather than deep-copying its fitted state.

Q7: How does Prototype help avoid "class explosion"? Instead of writing a new subclass for every configuration variant (as you might with Factory Method), you keep one well-configured prototype instance and clone-and-tweak it per variant at runtime — no new classes required (Section 12).

Q8: What are the risks of blindly deep-copying an object? Objects holding external resources — open file handles, network connections, database sessions — usually shouldn't be duplicated at all; a naive deep copy can break those resources or create unwanted duplicates of things meant to be singular. Custom `__deepcopy__` logic (or deliberately excluding those fields) is the fix.

Q9: How is Prototype different from Factory Method? Factory Method builds a new object from scratch via its constructor, deciding which class to use. Prototype skips construction entirely for a new object by copying an existing one's state — no constructor call, no class decision, just duplication.

16. One-Page Cheat Sheet

- **Definition:** create new objects by copying an existing, already-configured prototype instead of building from scratch.

- **Three participants:** Prototype (declares cloning), ConcretePrototype (implements it), Client (asks for a clone without knowing the concrete class).
 - **Idiomatic Python:** `copy.deepcopy()` already implements this — you rarely need to hand-roll a `clone()` protocol.
 - **Critical fact:** cloning does NOT re-run `__init__` — this is exactly why it's faster than constructing for expensive objects.
 - **The #1 gotcha:** shallow copy (`copy.copy()`) shares nested mutable state with the original; deep copy (`copy.deepcopy()`) makes everything independent. Mixing these up is the most common bug.
 - **Advanced move:** override `__deepcopy__` to deep-copy some fields while deliberately sharing expensive, safe-to-share resources.
 - **Registry variant:** a dictionary of named prototype instances, cloned on request — same idea as registry-based factories, but cloning objects instead of instantiating classes.
 - **Avoids class explosion:** clone-and-tweak a runtime object instead of writing a new subclass per variant.
 - **Real examples:** `list.copy()`/`dict.copy()`, `sklearn.base.clone()`, Java's `Cloneable`.
 - **Use when:** construction is expensive or you need many near-identical objects. **Be careful when:** objects hold external resources that shouldn't be duplicated.
-

That completes the full set of five creational patterns: Singleton (exactly one instance), Factory Method (one product, subclass decides), Abstract Factory (a matched family of products, one factory decides all of them), Builder (one complex object, assembled step by step), and Prototype (copy an existing object instead of building a new one). In a system design interview, being able to say — out loud, in one sentence — which of these five fits a given object-creation problem, and why the other four don't, is a stronger signal than reciting any single pattern perfectly.